



UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

INGENIERÍA EN DESARROLLO Y GESTION DE SOFTWARE

ASIGNATURA: DESARROLLO WEB PROFESIONAL

8º CUATRIMESTRE GRUPO “A”

PROTECCIÓN DE RUTAS Y RECURSOS POR ROL

PROFR. JOSE MIGUEL CARRERA PACHECHO

LILIA HERNÁNDEZ TUN

1. ¿QUÉ ES UN MIDDLEWARE?

Es un software que actúa como un intermediario entre distintas aplicaciones, servicios o componentes dentro de un sistema, cuyo propósito es facilitar la comunicación, integración y gestión de datos entre diferentes entornos.

TIPOS DE MIDDLEWARE	
Mensajería	Facilita la comunicación asincrónica entre aplicaciones a través de mensajes. Es ideal cuando los sistemas no están activos al mismo tiempo o cuando se necesita desacoplar procesos.
Orientado a la base de datos	Conecta las aplicaciones con los sistemas de gestión de bases de datos (DBMS), permitiendo el acceso a los datos sin que las aplicaciones tengan que interactuar directamente con las bases de datos.
De integración	Se utiliza para interconectar aplicaciones que operan en plataformas o lenguajes de programación distintos, asegurando que puedan comunicarse entre sí sin problemas. A través de interfaces comunes, elimina la necesidad de adaptar cada sistema al otro, simplificando la integración para lograr que los sistemas trabajen juntos de manera coherente.
De seguridad	Agrega una capa de protección esencial, asegurando que las aplicaciones estén blindadas contra accesos no autorizados. Ofrece funciones como autenticación, autorización y cifrado de datos, protegiendo contra posibles vulnerabilidades de seguridad.

2. ¿CÓMO SE PROTEGEN RUTAS EN BACKEND?

Proteger las rutas en el backend es fundamental para asegurar que solo los usuarios autorizados accedan a recursos específicos.

EL USO DE MIDDLEWARES: La estrategia principal en el backend es el uso de Middlewares. Estos son funciones que se ejecutan de manera secuencial antes de que la petición llegue a la lógica de negocio.

- **Interceptación:** El middleware analiza los encabezados (headers) de la petición buscando un token de acceso (comúnmente un **JWT - JSON Web Token**).
- **Validación:** Si el token no existe, está expirado o la firma es incorrecta, el servidor detiene la petición inmediatamente y responde con un error 401 Unauthorized.

AUTENTICACIÓN VS. AUTORIZACIÓN: Las fuentes enfatizan que no basta con saber "quién" es el usuario, sino también "qué" puede hacer:

- **Autenticación:** Verifica la identidad (ej. login con éxito).
- **Autorización (RBAC):** Una vez dentro, el backend debe verificar los roles. Por ejemplo, si una ruta es `/api/admin/delete-user`, el middleware debe confirmar que el usuario autenticado tiene el rol de "Administrador". Si es un usuario estándar, el servidor responde 403 Forbidden.

VALIDACIÓN Y SANITIZACIÓN DE DATOS: La protección de rutas también implica proteger la base de datos de ataques directos:

- **Prevención de Inyección SQL:** Las rutas deben estar programadas para validar y limpiar cualquier dato de entrada. Nunca se debe confiar en lo que el usuario envía en la URL o el cuerpo del mensaje.
- **WAF (Web Application Firewall):** Se recomienda como una capa externa que filtra el tráfico malicioso (como ataques DDoS o intentos de inyección) antes de que siquiera lleguen a las rutas de tu servidor.

IMPLEMENTACIÓN DE CAPAS (ESTRATEGIA DE DEFENSA EN PROFUNDIDAD): Es importante notar que el backend protege las rutas mediante estas capas:

- **Capa de Red:** HTTPS para cifrar la comunicación y evitar que los tokens sean robados.

- **Capa de Acceso:** Validación de tokens JWT y manejo de sesiones.
- **Capa de Aplicación:** Middlewares de roles y permisos específicos por ruta.
- **Capa de Datos:** Validación de esquemas y sanitización para evitar corrupción de la base de datos.

3. ¿QUÉ ERRORES DE SEGURIDAD SON COMUNES?

Confiar en el Frontend

Este es el error número uno. Muchos desarrolladores creen que porque ocultaron un botón en React o protegieron una ruta con React Router, la aplicación es segura.

- **El error:** No validar los permisos de nuevo en el Backend.
- **Consecuencia:** Un atacante puede usar herramientas como Postman o Burp Suite para enviar peticiones directamente a la API, saltándose toda la interfaz de usuario.

Gestión Deficiente de Tokens (JWT)

Los JSON Web Tokens son potentes, pero se implementan mal con frecuencia:

- **Almacenamiento inseguro:** Guardar tokens sensibles en localStorage. Esto los hace vulnerables a ataques **XSS** (Cross-Site Scripting), donde un script malicioso puede robar el token.
- **Falta de expiración:** Tokens que nunca vencen. Si un token es robado, el atacante tiene acceso perpetuo.
- **No verificar la firma:** Configurar el backend para que acepte tokens sin validar la clave secreta (secret key).

Inyección (SQL y NoSQL)

- **El error:** Concatenar variables directamente en las consultas a la base de datos (ej. `SELECT * FROM users WHERE id = ' + id + '`).

- **Consecuencia:** Un atacante puede escribir `1' OR '1'='1` en un campo de texto y obtener acceso a toda la tabla de usuarios.
- **Solución:** Usar **Consultas Preparadas** o un ORM (como Sequelize o Mongoose).

Broken Access Control (Control de Acceso Roto)

Ocurre cuando un usuario puede acceder a recursos que no le pertenecen simplemente cambiando un ID en la URL.

- **Ejemplo:** Estás logueado como el usuario 10, pero si cambias la URL a `/api/pedidos/11`, el sistema te muestra los datos del otro usuario porque el backend solo revisó que estuvieras "logueado", pero no que fueras el **dueño** de ese pedido.

Exposición de Datos Sensibles en Errores

- **El error:** Dejar activo el modo "debug" en producción o devolver el error completo de la base de datos al cliente.
- **Consecuencia:** El atacante recibe información sobre la estructura de tus tablas, versiones de software o rutas de archivos del servidor, lo que le da un mapa para el siguiente ataque.

Configuración Incorrecta de CORS

El **CORS** (Cross-Origin Resource Sharing) es una medida de seguridad del navegador.

- **El error:** Configurar `Access-Control-Allow-Origin: *` (permitir a todos).
- **Consecuencia:** Cualquier sitio web malicioso puede intentar hacer peticiones a tu backend en nombre del usuario si este tiene la sesión abierta

4. **¿CÓMO SE VALIDA ACCESO POR ROL?** El Control de Acceso Basado en Roles (RBAC, por sus siglas en inglés) es un método de regulación de acceso a recursos en el cual los permisos se asignan a roles específicos, y no directamente a los usuarios individuales.

Componentes Clave de la Validación

Para que una validación por rol sea efectiva, deben existir tres elementos vinculados:

- **Permiso (Scope/Ámbito):** Es la unidad mínima de acceso (ej. read:orders, write:products). Define qué acción se puede realizar sobre qué recurso.
- **Rol:** Un conjunto de permisos predefinidos. Por ejemplo, el rol "Vendedor" puede tener los permisos de lectura y creación de pedidos, pero no de eliminación.
- **Usuario:** La entidad final a la que se le asigna uno o más roles. Al recibir un rol, el usuario hereda automáticamente todos los permisos asociados a este.

El Proceso de Validación (Flujo Lógico)

Para que el sistema permita o deniegue el acceso, se sigue un flujo de validación técnica que ocurre usualmente en el backend:

1. **Autenticación y Emisión de Claims:** Cuando el usuario inicia sesión, el sistema de identidad genera un token (JWT). Este token contiene "claims" o declaraciones que incluyen los roles asignados al usuario.
2. **Petición al Recurso:** El usuario intenta acceder a una ruta protegida (ej. /api/delete-user).
3. **Extracción de Roles:** El servidor recibe el token, lo decodifica y extrae la lista de roles del usuario.
4. **Verificación de Correspondencia:** El sistema compara los roles del usuario con los requeridos por la ruta. Si la ruta exige el rol admin y el usuario solo posee el rol user, el acceso es rechazado con un código **403 Forbidden**.

Beneficios de Validar por Roles según las Fuentes

- **Eficiencia Administrativa:** No es necesario cambiar los permisos de cada empleado individualmente cuando cambian de puesto; basta con cambiar su rol.

- **Principio de Menor Privilegio:** Como señala Cloudflare, el RBAC ayuda a asegurar que los empleados solo tengan acceso a la información estrictamente necesaria para su trabajo, reduciendo el riesgo de brechas de seguridad.
- **Cumplimiento (Compliance):** Facilita las auditorías de seguridad, ya que es más sencillo demostrar quién tiene acceso a qué basándose en una estructura jerárquica clara.

ENTREGABLES

Middleware de Autenticación Global (authMiddleware.js)

Se implementó un middleware de seguridad basado en el estándar JWT (JSON Web Tokens). Este componente actúa como la primera barrera de defensa en el backend, interceptando las peticiones antes de que lleguen a los datos sensibles.

Extracción de Credenciales (Línea 8): El sistema busca el token en el encabezado de Authorization.

- **Validación de Presencia (Líneas 11-13):** Si el token no existe, se bloquea el acceso con un código 401 (Acceso denegado), evitando accesos anónimos.
- **Limpieza de Token (Líneas 17-20):** Se implementó lógica para manejar el formato Bearer, asegurando la compatibilidad con diferentes clientes (Frontend o Postman).
- **Verificación Criptográfica (Línea 23):** Se usa una JWT_SECRET para comprobar que el token no ha sido manipulado.
- **Inyección de Identidad (Línea 27):** Si el token es válido, los datos del usuario (incluyendo su rol) se inyectan en el objeto req.user. Esto permite que los siguientes controladores sepan exactamente quién está haciendo la petición sin tener que volver a consultar la base de datos.

"portero" (middleware) nunca dejará pasar al usuario a las rutas protegidas como el Dashboard o el Generador de QR.

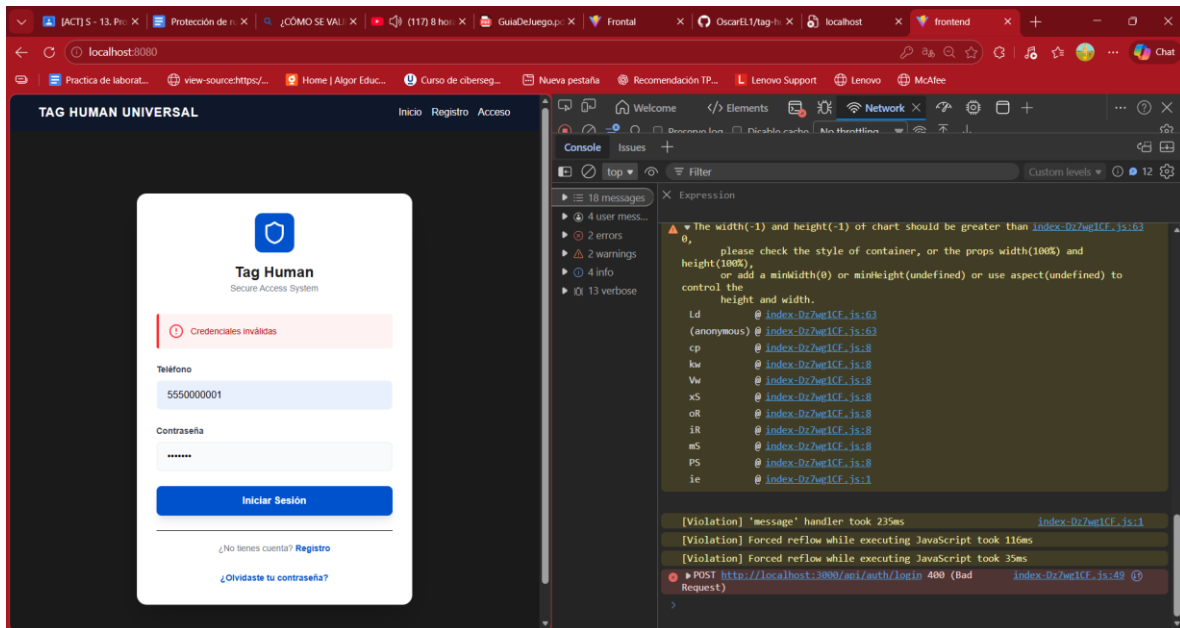


Ilustración 2 Validación de Credenciales y Manejo de Errores (400 Bad Request)

Implementación de Rutas Protegidas (Capa de Aplicación)

- **La Inyección del Middleware (Línea 41):** Al insertar el parámetro auth en la ruta `router.get('/me', auth, ...)`, estás aplicando una política de Denegación por Defecto.
- **Seguridad en la Consulta (Línea 47):** Observa cómo usas `WHERE id = $1` y pasas `[req.user.id]`. Esto evita ataques de **Inyección SQL** (al usar parámetros preparados) y ataques de IDOR (porque no confías en lo que el usuario manda en la URL, sino en el ID que viene "sellado" dentro del token JWT que el middleware decodificó).
- **Validación de Datos (Líneas 17-28):** Muestra que no solo proteges el acceso, sino también la integridad de los datos usando `check()` para validar teléfonos y formatos de placas de vehículos.

- **Cierre de Brecha:** Se demuestra que, aunque un atacante conozca la estructura de las rutas de la API (por ejemplo, mediante ingeniería inversa del código fuente), el sistema de **Identidad Digital** basado en JWT impide la extracción de información sin una firma criptográfica válida.

Conclusión Técnica: El backend de **Tag Human Universal** implementa una arquitectura de "Confianza Cero" (Zero Trust). No confía en el origen de la petición, sino en la validez del token, garantizando que los datos sensibles permanezcan privados incluso si el cliente (frontend) es comprometido o evadido.

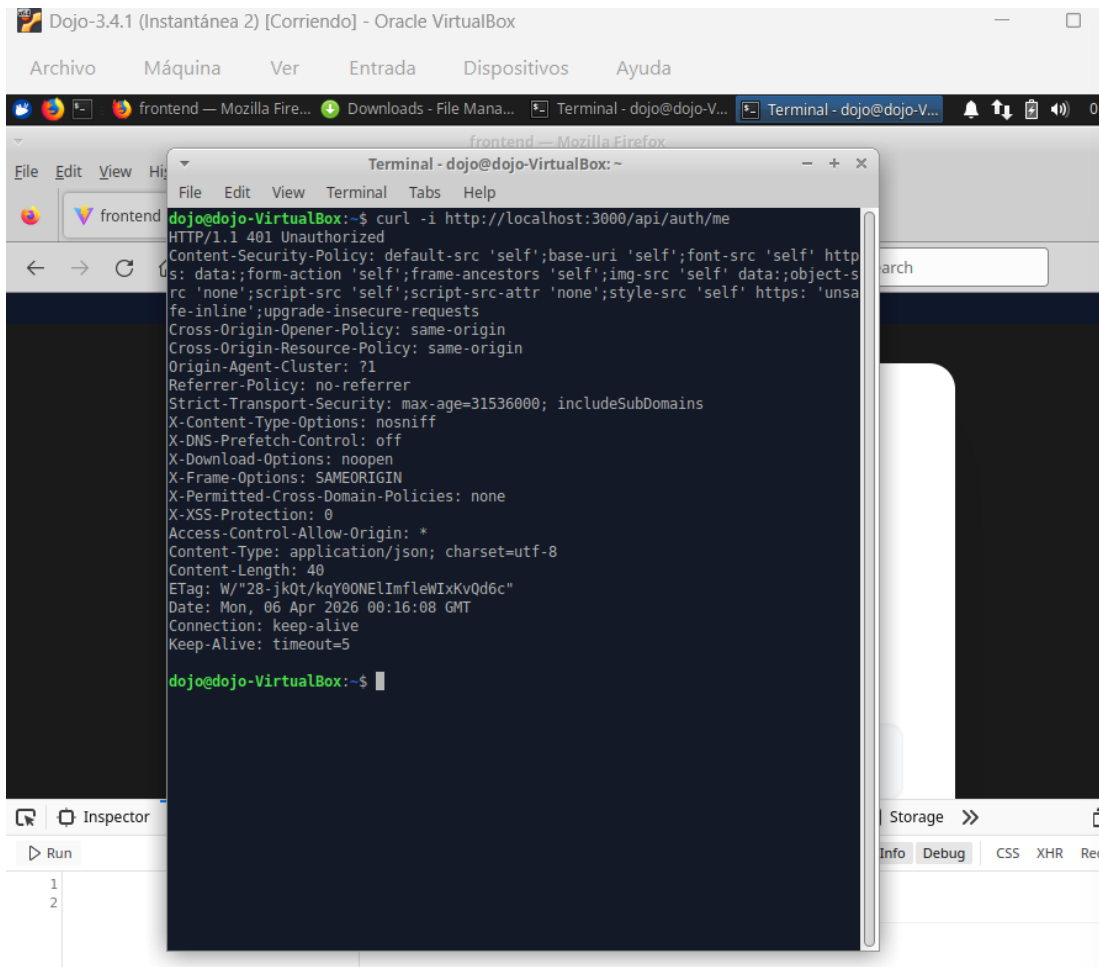


Ilustración 4 Prueba de Seguridad en Capa de Red (API Shielding)

Prueba de Protección de Endpoints (Filtro de Servidor Web)

- Acción: Se intentó realizar una petición POST maliciosa directamente a la raíz del servidor de interfaz (puerto 8080).
- Resultado: El servidor Nginx respondió con un código 405 Not Allowed.
- El sistema está protegido contra intentos de inyección de datos en rutas no autorizadas. El servidor web está configurado correctamente para denegar métodos de escritura en rutas estáticas, lo que previene ataques de manipulación de API accidentales o malintencionados.

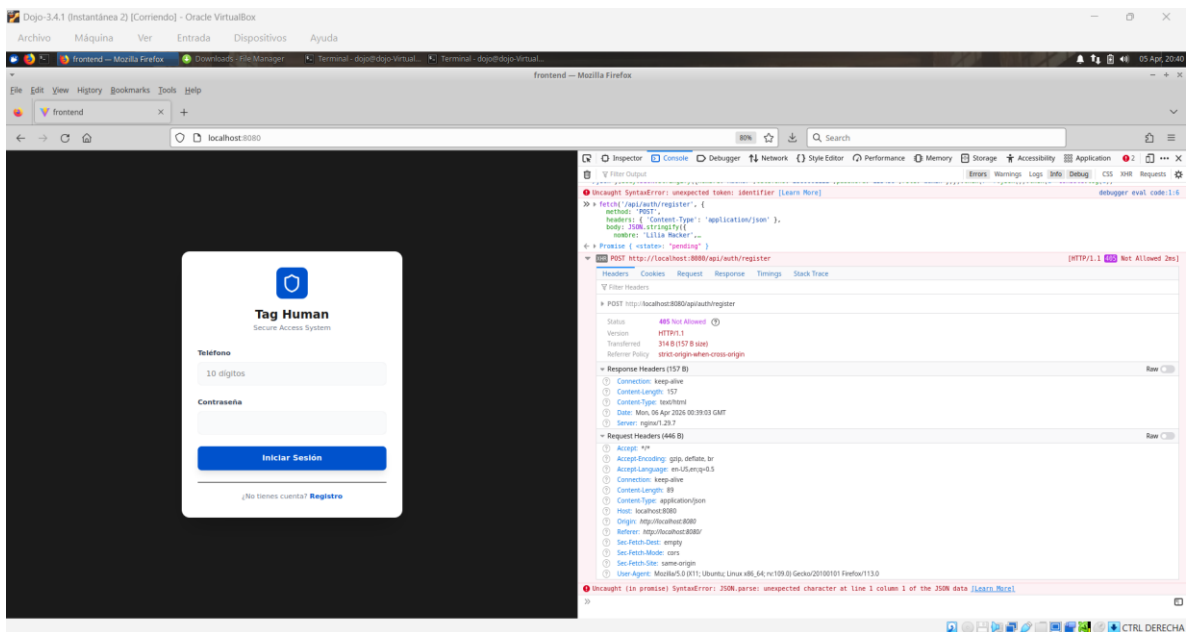


Ilustración 5 Prueba de Protección de Endpoints

Prueba de Restricción de Métodos HTTP (GET vs POST)

- Acción: Se intentó acceder directamente vía URL (navegador) al endpoint de registro en el puerto 3000.
- Resultado: El servidor devolvió un error 404 Not Found y el mensaje Cannot GET.
- Se confirma que el Backend está configurado correctamente para no exponer información ante peticiones de tipo GET en rutas críticas. El sistema solo responde a peticiones POST debidamente estructuradas, lo que reduce la superficie de ataque ante escaneos automáticos de vulnerabilidades.

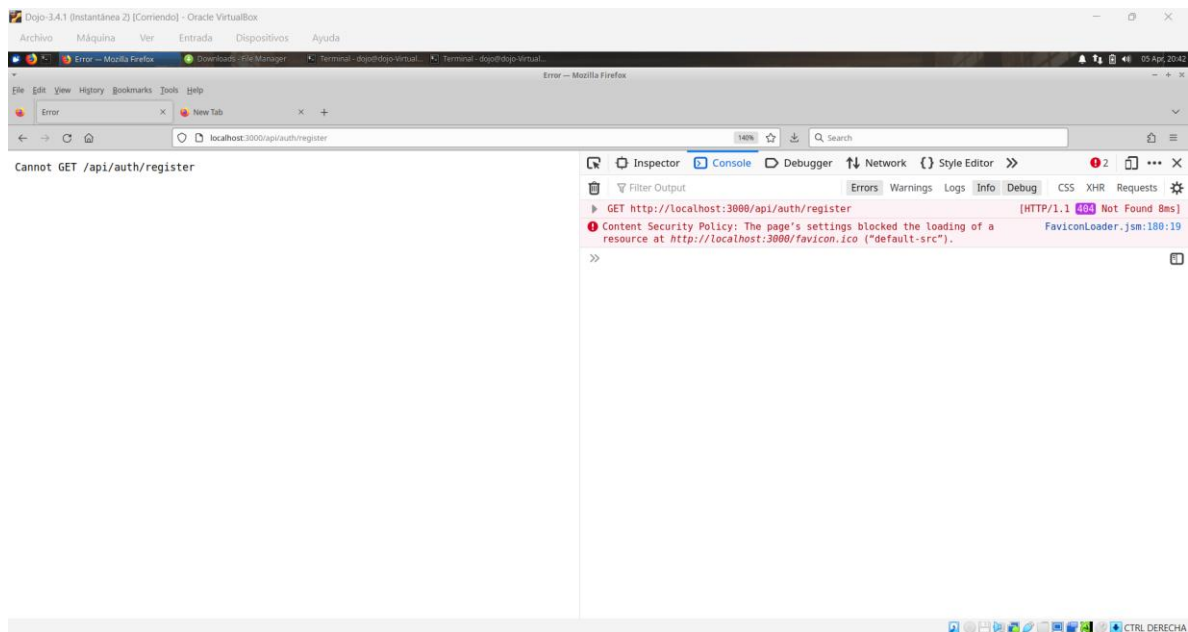


Ilustración 6 Prueba de Restricción de Métodos HTTP (GET vs POST)

REFERENCIAS

- **Cloudflare.** (s.f.). *¿Qué es el control de acceso basado en roles (RBAC)?* Cloudflare. <https://www.cloudflare.com/es-es/learning/access-management/role-based-access-control-rbac/>
- **DeintaSC.** (s.f.). *Qué es el middleware: Definición y ejemplos.* Deinta Seguridad y Comunicaciones. <https://www.deintasc.com/que-es-el-middleware-definicion-y-ejemplos/>
- **Logto Docs.** (s.f.). *Control de acceso basado en roles (RBAC) (Role-based access control).* Logto Documentation. <https://docs.logto.io/es/docs/recipes/rbac/>
- **React Router.** (s.f.). *Rutas Privadas con React Router: Protegiendo Rutas de Aplicación.* Documentación oficial de React Router. <https://reactrouter.com/en/main/start/overview>
- **Treelab.** (s.f.). *¿Qué es un middleware y cómo se usa en el desarrollo de software?* Blog Treelab: Tecnología y Diseño. <https://blog.treelab.io/que-es-un-middleware-y-como-se-usa/>